TikuOS MPU Write-Protection Wrappers

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 6, 2026

Outline

The Problem: Unprotected FRAM

Without MPU protection:

- Wild pointers can corrupt FRAM silently
- Runaway code can overwrite calibration data
- ISR bugs can damage persistent state
- Stack overflows can reach FRAM regions
- Corruption persists across reboots

TikuOS MPU wrappers:

- Default: FRAM is read+execute, **no write**
- Explicit unlock/lock for intentional writes
- Scoped write helper disables interrupts
- Per-segment permission control
- Password-protected hardware registers

FRAM Makes It Worse

Unlike SRAM, FRAM is non-volatile — a single wild write persists across power cycles. A corrupted calibration value stays corrupted until explicitly fixed.

Defense in Depth

The MPU acts as a hardware safety net — even if software has a bug, it cannot accidentally modify FRAM without going through the controlled unlock sequence.

MPU Register Architecture

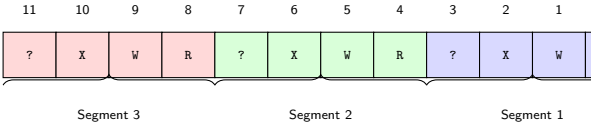
Three key registers:

Register	Purpose
`MPUCTL0`	Control: password (upper byte), enable bit, lock bit
`MPUCTL1`	Violation flags (which segment was violated)
`MPUSAM`	Segment Access Mode: R/W/X permissions per segment

Password Protection

Every write to `MPUCTL0` must include `0xA500` in the upper byte. Without it, the hardware ignores the write (or triggers a reset). This prevents wild pointers from accidentally changing memory protection.

MPUSAM bit layout:



Each segment has 3 permission bits (R/W/X) in a 4-bit nybble.

Default value `0x0555`**:**
0x5 = 0b0101 = R+X (no W) per segment.
Three segments: `0x0555`.

Unlock value `0x0222`**:**
ORing this sets the W bit in all three segments.

MPU Types (kernel/memory/tiku_mem.h)

Segment identifiers

```
typedef enum {
    TIKU_MPU_SEG1 = 0,
    TIKU_MPU_SEG2 = 1,
    TIKU_MPU_SEG3 = 2
} tiku_mpu_seg_t;
```

Permission flags

```
typedef enum {
    TIKU_MPU_READ     = 0x01,
    TIKU_MPU_WRITE    = 0x02,
    TIKU_MPU_EXEC     = 0x04,
    TIKU_MPU_RD_WR    = 0x03,
    TIKU_MPU_RD_EXEC  = 0x05,
    TIKU_MPU_ALL      = 0x07
} tiku_mpu_perm_t;
```

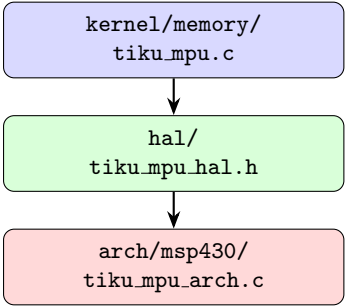
Scoped write callback

```
typedef void (*tiku_mpu_write_fn)(
    void *ctx);
```

Design Choices

- Permission flags map directly to hardware bit positions
- Compound permissions (`RD_WR`, `RD_EXEC`) are pre-defined for convenience
- Callback takes an opaque `void *ctx` so the caller can pass any data without globals

Three-Layer Architecture



Platform-independent logic:
init, set_permissions, unlock, lock, scoped_write

Routes to arch header;
declares HAL function signatures

MSP430 hardware access:
MPUCTL0 password, MPUSAM, intrinsics

Why This Layering?

The kernel MPU code (`tiku_mpu.c`) contains only bit manipulation and sequencing logic — no `#include <msp430.h>`, no register names, no intrinsics. Porting to a new platform means writing a new arch file; the kernel code stays untouched.

HAL Interface

hal/tiku_mpu_hal.h — Platform functions

```
/* Read the segment-access-mode register */
uint16_t tiku_mpu_arch_get_sam(void);

/* Write new permissions (handles password/unlock internally) */
void tiku_mpu_arch_set_sam(uint16_t sam);

/* Read MPU control register (check enable bit) */
uint16_t tiku_mpu_arch_get_ctl(void);

/* Interrupt control for critical sections */
void tiku_mpu_arch_disable_irq(void);
void tiku_mpu_arch_enable_irq(void);
```

Kernel Calls HAL

The kernel never writes to hardware registers directly. It calls `tiku_mpu_arch_set_sam()` which handles the password sequence internally.

Host Testing

On the host, these functions are stubbed with plain variables. Tests can inspect the resulting state without real MPU hardware.

MSP430 Arch Implementation

arch/msp430/tiku_mpu_arch.c

```c
#include "tiku_mpu_arch.h"
#include <msp430.h>

uint16_t tiku_mpu_arch_get_sam(void) {
    return MPUSAM;
}

void tiku_mpu_arch_set_sam(uint16_t sam) {
    MPUCTL0 = MPUPW;            /* Unlock with password (0xA500) */
    MPUSAM  = sam;              /* Set new permissions */
    MPUCTL0 = MPUPW | MPUENA;  /* Re-enable MPU */
}

uint16_t tiku_mpu_arch_get_ctl(void) {
    return MPUCTL0;
}

void tiku_mpu_arch_disable_irq(void) { __disable_interrupt(); }
void tiku_mpu_arch_enable_irq(void)  { __enable_interrupt(); }
```

All Hardware Access in One File

The password (MPUPW), enable bit (MPUENA), and MSP430 intrinsics are confined to this single arch file

tiku_mpu_init()

```c
void tiku_mpu_init(void);
```

- Sets all three segments to **read + execute, no write** (0x0555)
- Called by `tiku_mem_init()` at boot — first thing before any other subsystem
- After this call, any attempt to write to FRAM triggers an MPU violation

kernel/memory/tiku_mpu.c

```c
#define MPU_SAM_DEFAULT   0x0555  /* R+X on all 3 segments */

void tiku_mpu_init(void) {
    tiku_mpu_arch_set_sam(MPU_SAM_DEFAULT);
}
```

Boot Sequence

`tiku_mem_init()` calls `tiku_mpu_init()` first, then `tiku_mem_arch_init()`. FRAM is write-protected before any other memory subsystem initialization runs.

tiku_mpu_unlock_fram() / tiku_mpu_lock_fram()

Unlock: enable writes

```
uint16_t tiku_mpu_unlock_fram(void)
{
    uint16_t saved =
        tiku_mpu_arch_get_sam();

    tiku_mpu_arch_set_sam(
        saved | 0x0222);

    return saved;
}
```

- Saves current SAM state
- ORs in write bit for all segments
- Returns saved state for later restore

Lock: restore state

```
void tiku_mpu_lock_fram(
    uint16_t saved_state)
{
    tiku_mpu_arch_set_sam(saved_state);
}
```

- Restores SAM to pre-unlock value
- Removes write permission
- Safe to call when already locked

Usage Pattern

```
saved = unlock();
// ...  write to FRAM ...
lock(saved);
```

tiku_mpu_scoped_write()

```c
void tiku_mpu_scoped_write(tiku_mpu_write_fn fn, void *ctx);
```

kernel/memory/tiku_mpu.c

```c
void tiku_mpu_scoped_write(tiku_mpu_write_fn fn, void *ctx) {
    uint16_t saved;

    tiku_mpu_arch_disable_irq();   /* 1. Disable interrupts */
    saved = tiku_mpu_unlock_fram(); /* 2. Unlock FRAM */

    fn(ctx);                        /* 3. Execute write function */

    tiku_mpu_lock_fram(saved);      /* 4. Relock FRAM */
    tiku_mpu_arch_enable_irq();    /* 5. Re-enable interrupts */
}
```

Why Disable Interrupts?

While FRAM is unlocked, any ISR has write access. A bug in an ISR could corrupt persistent data. Disabling interrupts ensures only the caller's function writes to FRAM.

Keep It Short

While interrupts are disabled, hardware events queue up. The write function must be brief — long operations should be split across multiple scoped_write calls.

tiku_mpu_set_permissions()

```
void tiku_mpu_set_permissions(tiku_mpu_seg_t seg, tiku_mpu_perm_t perm);
```

kernel/memory/tiku_mpu.c

```
void tiku_mpu_set_permissions(tiku_mpu_seg_t seg, tiku_mpu_perm_t perm) {
    uint16_t shift = (uint16_t)seg * 4U;
    uint16_t mask  = (uint16_t)0x07 << shift;
    uint16_t sam   = tiku_mpu_arch_get_sam();

    sam = (sam & ~mask) | (((uint16_t)perm & 0x07) << shift);
    tiku_mpu_arch_set_sam(sam);
}
```

- Modifies only the targeted segment's 3-bit permission field
- Other segments are left untouched
- Use when you need finer control than the all-segments unlock/lock

Example: Make segment 3 writable while keeping segments 1–2 locked:

```
tiku_mpu_set_permissions(TIKU_MPU_SEG3, TIKU_MPU_RD_WR);
/* ... write to segment 3 FRAM region ... */
tiku_mpu_set_permissions(TIKU_MPU_SEG3, TIKU_MPU_RD_EXEC);
```

End-to-End: Persisting Sensor Calibration

```c
/* Context for the scoped write callback */
typedef struct {
    tiku_persist_store_t *store;
    const uint8_t *data;
    tiku_mem_arch_size_t len;
} cal_write_ctx_t;

static void write_cal(void *arg) {
    cal_write_ctx_t *ctx = (cal_write_ctx_t *)arg;
    tiku_persist_write(ctx->store, "cal", ctx->data, ctx->len);
}

void save_calibration(tiku_persist_store_t *store,
                      const uint8_t *cal_data, tiku_mem_arch_size_t len)
{
    cal_write_ctx_t ctx = { store, cal_data, len };

    /* scoped_write: disable IRQ -> unlock FRAM -> write -> lock -> enable IRQ */
    tiku_mpu_scoped_write(write_cal, &ctx);
}
```

Safety Guarantees

FRAM is writable only during the `write_cal` callback. Interrupts are disabled so no ISR can interfere.

Manual Unlock/Lock for Batched Writes

```c
void batch_persist_update(tiku_persist_store_t *store) {
    uint16_t saved;
    uint8_t cfg_data[] = {0x01, 0x02};
    uint8_t cal_data[] = {0xAA, 0xBB, 0xCC};

    /* Batch multiple FRAM writes in one unlock window */
    saved = tiku_mpu_unlock_fram();

    tiku_persist_write(store, "cfg", cfg_data, sizeof(cfg_data));
    tiku_persist_write(store, "cal", cal_data, sizeof(cal_data));

    tiku_mpu_lock_fram(saved);
}
```

Trade-off: Batching vs Safety

Manual unlock/lock avoids per-write overhead but leaves FRAM unprotected for longer. Use `scoped_write` for single writes; manual unlock for known-safe batches where the caller controls the critical section.

Module File Organization

Layer	File	Role
Kernel (portable)	`kernel/memory/tiku_mem.h`	Unified API: arena + persist + MPU types
	`kernel/memory/tiku_mem.c`	Arena allocator, module init
	`kernel/memory/tiku_mpu.c`	MPU logic: init, unlock, lock, scoped write
HAL	`hal/tiku_mem_hal.h`	Routes to arch memory header
	`hal/tiku_mpu_hal.h`	Routes to arch MPU header
Arch (MSP430)	`arch/msp430/tiku_mpu_arch.h`	MPU arch function declarations
	`arch/msp430/tiku_mpu_arch.c`	MPUCTL0/MPUSAM register access
Tests	`tests/test_tiku_mem.c`	25 test cases (arena + persist + MPU)

Porting to a New Architecture

Only the arch files change:

1. Implement `tiku_mpu_arch_get_sam()` / `set_sam()` for your MPU hardware
2. Implement `disable_irq()` / `enable_irq()` for your interrupt controller
3. The kernel MPU logic and all tests remain untouched

API at a Glance

Function	Returns	Complexity	Description
`tiku_mpu_init`	`void`	O(1)	Set default FRAM protection
`tiku_mpu_set_permissions`	`void`	O(1)	Set permissions on one segment
`tiku_mpu_unlock_fram`	`uint16_t`	O(1)	Enable writes, return saved state
`tiku_mpu_lock_fram`	`void`	O(1)	Restore saved protection state
`tiku_mpu_scoped_write`	`void`	O(1)+fn	IRQ-safe unlock/write/lock

Safety Guarantees

- FRAM write-protected by default
- Hardware password prevents accidental unlock
- Scoped write disables interrupts
- Lock/unlock are idempotent

Design Choices

- Platform-independent kernel code
- All register access in arch layer
- HAL routes to correct platform
- Host-testable with stub registers

Test Configuration

Host-native (no hardware):

```
gcc -std=c99 -Wall -Wextra -I. \
  tests/test_tiku_mem.c \
  kernel/memory/tiku_mem.c \
  kernel/memory/tiku_persist.c \
  kernel/memory/tiku_mpu.c \
  -o test_tiku_mem \
  && ./test_tiku_mem
```

Host mode provides stub implementations of all HAL functions — plain variables mimic MPU registers so the test can inspect resulting state without hardware.

Host HAL stubs in test file

```
/* Stub MPU registers */
static uint16_t stub_mpuctl0;
static uint16_t stub_mpusam;

uint16_t tiku_mpu_arch_get_sam(void)
{ return stub_mpusam; }

void tiku_mpu_arch_set_sam(uint16_t sam)
{
    stub_mpuctl0 = 0xA500;
    stub_mpusam  = sam;
    stub_mpuctl0 = 0xA500 | 0x0001;
}

void tiku_mpu_arch_disable_irq(void)
{ /* no-op */ }
void tiku_mpu_arch_enable_irq(void)
{ /* no-op */ }
```

MPU Test Cases

#	Test Name	What It Verifies
21	Init defaults	SAM = 0x0555 (all R+X); CTL enable bit set
22	Unlock / Lock	Unlock returns saved state (0x0555); SAM becomes 0x0777; lock restores 0x0555
23	Set permissions	Changing one segment leaves others untouched
24	Scoped write	Callback invoked with write bits set; SAM relocked after return
25	Idempotency	Lock when locked is no-op; double unlock returns correct states; original saved state restores correctly

Combined Test Binary

MPU tests (5 groups, 14 assertions) run alongside arena (9 tests) and persistent store (11 tests) in a single binary — 136 total assertions, all passing on host GCC.

Test Deep Dive: Scoped Write

test_tiku_mem.c — Scoped write test

```c
typedef struct {
    int called;
    uint16_t sam_during;
} scoped_write_ctx_t;

static void scoped_write_cb(void *arg) {
    scoped_write_ctx_t *ctx = (scoped_write_ctx_t *)arg;
    ctx->called = 1;
    ctx->sam_during = tiku_mpu_arch_get_sam();  /* capture state during write */
}

void test_mpu_scoped_write(void) {
    scoped_write_ctx_t ctx = { 0, 0 };

    tiku_mpu_init();
    tiku_mpu_scoped_write(scoped_write_cb, &ctx);

    TEST_ASSERT(ctx.called == 1);                          /* callback was invoked */
    TEST_ASSERT((ctx.sam_during & 0x0222) == 0x0222);      /* write bits set during */
    TEST_ASSERT(tiku_mpu_arch_get_sam() == 0x0555);        /* locked after return */
}
```

What This Proves

Summary

MPU Wrapper Design

- Hardware FRAM write-protection
- Default: read+execute, no write
- Explicit unlock/lock pattern
- Scoped write with IRQ protection
- Per-segment fine-grained control
- Password-protected registers
- Platform-independent kernel code
- All hardware access in arch layer

Test Coverage

- 5 MPU test groups (14 assertions)
- Runs on host GCC with stub registers
- Covers: init, unlock/lock, per-segment, scoped write, idempotency
- Part of unified memory test binary (136 total)

Part of the Memory Module

The MPU wrappers extend `tiku_mem.h` alongside the arena allocator and persistent store — same header, same HAL pattern, same test binary.

Thank You

TikuOS MPU Write-Protection Wrappers

FRAM Safety for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
API Header:	`kernel/memory/tiku_mem.h`
MPU Logic:	`kernel/memory/tiku_mpu.c`
Tests:	`tests/test_tiku_mem.c`